

# CSE5IDP Week 8 — Project Metrics, AI-Assisted Development and Testing

Lecture summary · Scott Mann · Semester 2, 2026

Lecture delivered in Week 8 (first session after the mid-semester break)

## At a glance

---

The session ran roughly 1 h 45 min, with a five-minute break at about 3:21 pm. An employability workshop was running in parallel, so the face-to-face room was small; Helsin, Jaya, Duroz and Nisha joined online. Scott covered three blocks:

1. **Project metrics** (continuing Topic 3 from Week 7) — size-oriented versus function-oriented measurement, object-oriented complexity metrics, and control charts as career evidence.
2. **AI and the developer's value proposition** — why “lines of code” no longer means much, what employers now keep, and how to *prove* AI-assisted capability with tracked metrics.
3. **Testing in 2026** (Milestone 3) — testing versus debugging, the V&V diagram and where AI fits, TDD with user-story test tables, CI/CD automation, and why security sign-off stays human.

“Effective reporting” (the final handover document and presentation) was deferred to Week 9.

*Re-establish productivity quickly after the break — teams that did nothing over the break can lose a full week getting back up to speed, and the semester is moving fast.*

## Housekeeping

---

- **Team responsibility tracker.** Scott posted an announcement on the LMS with an Excel spreadsheet for tracking responsibilities sprint by sprint. Teams with performance or responsibility issues (which tend to surface as poor team sentiment) should use it.
- **Week 8 is a pivot point.** Many teams have had little contact with their project owners so far. Reach out now to confirm direction before the handover phase.
- **Scott visiting the labs** the following afternoon (after class, around 1:30 pm).

# 1. Project metrics

## Recap of Week 7

Process metrics (burndown, burn-up, committed vs completed, work by tech category, velocity) track team productivity over time and can be exported from Jira and plotted. The pattern to aim for is steadily increasing throughput across sprints; a zig-zag is not a failure but may point at responsibility, scoping or estimation problems.

## Project metrics and the right attitude

Project metrics affect **time frame and budget**. Scott repeated the framing from last week:

- Metrics are for **tracking progress, not calling out individuals**. Apply common sense and sensitivity; they are an information-gathering process.
- Regular, consistent data points are what make prediction possible — this dovetails with the per-sprint Scrum Master reports and feedback in labs.
- Poor numbers are **opportunities for improvement, not negatives**. “You can’t control the people; you can control your response to people.” How you respond is a marker of resilience, which employers value.
- **Do not fixate on a single data point or metric** — collate a suite.

## Size-oriented versus function-oriented measurement

**Size-oriented** metrics normalise by code volume, e.g. errors per thousand lines of code:

$$\text{Error density} = \frac{E}{\text{KLOC}}, \quad \text{Productivity} = \frac{\text{KLOC}}{\text{person-month}}$$

Scott’s argument against relying on these:

- Languages differ in verbosity; the same capability may take 50 lines in one language and 5 in another, and syntax and tooling keep improving even within one language.
- **Reuse and modular OO design shrink line counts** while hiding the fact that the code is more maintainable.
- **AI now generates much of the code**, so volume says little about quality or human contribution.

**Function-oriented** metrics measure delivered capability instead. The lecture walked through the function-point counting inputs (external inputs, outputs, inquiries, internal files, external interfaces) and the value-adjustment questions (“Does the system require reliable backup and recovery?”, “Is distributed processing required?”, “Are master files updated online?”, and so on). In the standard formulation these combine as

$$\text{FP} = \text{UFC} \times \left( 0.65 + 0.01 \sum_{i=1}^{14} F_i \right), \quad F_i \in \{0, 1, \dots, 5\}$$

where UFC is the weighted unadjusted count of the five measurement parameters and the  $F_i$  are the answers to the fourteen adjustment questions. Normalising by function points (for example  $E/\text{FP}$  or  $\text{FP}/\text{person-month}$ ) compares projects on outcome rather than syntax. The slide example

compared three projects (alpha, beta, gamma) on this basis, and a home-security scenario (users, sensors, monitoring, external dependencies) showed how interfaces and components feed the count.

## Object-oriented and design-level metrics

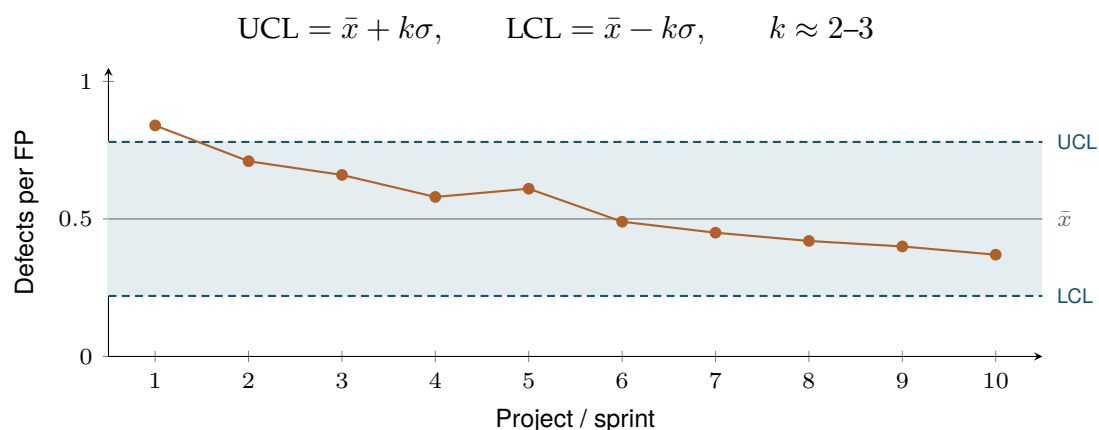
For OO code bases the lecture listed metrics that reflect complexity and design quality:

| Metric                                             | What it tells you                         |
|----------------------------------------------------|-------------------------------------------|
| Scenario scripts / use cases / user stories        | Size of the functional scope              |
| Number of key classes, subsystems                  | Size of the design                        |
| Weighted methods per class (WMC)                   | Per-class complexity                      |
| Depth of inheritance tree (DIT)                    | Hierarchy complexity                      |
| Number of children (NOC)                           | Breadth of the hierarchy                  |
| Coupling between objects (collaborators per class) | Coupling — a common 1st/2nd-year weakness |
| Lack of cohesion in methods (LCOM)                 | Cohesion of each class                    |

You do not count these by hand — IDEs and static-analysis tools compute them (Scott showed Visual Studio’s maintainability index, cyclomatic complexity, depth of inheritance and class coupling), and they can run automatically in a CI/CD pipeline.

## Control charts as a portfolio piece

The goal is to show **where you are, where you want to be, and that you operate inside a tolerance band** — for instance within a couple of standard deviations of a threshold:



Illustrative control chart: a developer’s defect rate tracked across projects, settling inside the tolerance band.

Scott’s pitch: the GitHub contribution heat-map shows *activity*, not *quality*. A “quality widget 2.0” tracking your metrics over two, three or four projects is a far stronger signal. Of 20–100 applicants for a dev role, 80–90% submit near-identical CVs; almost none show performance over time. Hiring is a risk, and longitudinal evidence is **risk reduction** for the employer — “you are the safest choice.”

## 2. AI and the developer's value proposition

### Who keeps their job

Developers doing simple, rules-based coding are being replaced because AI is good at applying rules. Management keeps developers who can:

- use AI tooling effectively to solve problems,
- **judge whether AI-generated output is good enough**, and
- combine AI with their own experience to find new opportunities (human-in-the-loop development).

Nobody is an “AI development expert” yet — everyone is two or three years into adapting. The programming language matters less than the ability to turn a human requirement into a working, tested system with whatever tools exist at the time.

### New metrics for AI-assisted work

Scott suggested metrics that do not yet exist in common practice:

- lines of AI-generated code produced per sprint or project;
- lines of AI-generated code **reviewed/debugged** per sprint (e.g. “200–500 lines per sprint”);
- errors found in AI-assisted code;
- time taken to evaluate a given amount of generated code;
- **mismatches between requirements and generated code** — a measure of how well you prompt and specify.

### The “AI assistance” workbook

A concrete homework-style recommendation:

1. In a spreadsheet, create a tab per project (including IDP, capstone, side projects, hackathons, open-source work — no project is too small).
2. On each tab record the metrics above.
3. Add a final aggregate tab that plots progress across all projects.
4. Put that chart on your CV, alongside a bolded **AI** section describing the AI-enabled tools you use and how you use them to scope, develop and test.

*“I’m not just a developer that can make a claim, I can actually back it up with evidence.”*

### Keep a learning diary

Record what worked and what didn’t with AI each week (e.g. “Codex did A, B, C well, but we spent four hours debugging code we didn’t understand; the prompt misread the requirement”). Scott prefers a handwritten notebook; a student pointed out that an electronic log can itself be fed to an AI later — Scott agreed both have merit. The point is to carry lessons across projects.

### Domain language and hybrid roles

AI-assisted generation is only as good as the instructions, and natural language is ambiguous. Models encode domain knowledge, so you must **speak the language of the domain** (healthcare, finance, environmental science, IoT, computer vision...) to prompt and evaluate effectively. **Hy-**

**brid roles** = domain knowledge + AI-assisted development. Two kinds of work are emerging:

- building from scratch as a hybrid engineer (what IDP teams are doing), and
- rescuing and extending systems that domain experts have **vibe-coded** — they load in a browser and hit a database, but nobody knows what is happening under the hood.

### Frontier models are often unnecessary

A student raised GPT-6 and how model quality can dip under high demand on subscriptions. Scott's points:

- Vendor claims (Anthropic, OpenAI, Google) must be **evaluated objectively against your own use cases**.
- **Most projects don't need frontier models**; models a year old are often fine. Newer models cost more, and chasing one or two "IQ points" is false economy.
- He recently moved his Azure AI Foundry workloads (about 120 models available) from frontier to mid-tier models; he could barely notice the difference in output and his running cost dropped substantially.
- A contact at Oracle tracks model/vendor cost versus quality side by side; the US approach chases raw performance, while the Chinese approach targets function and value. Scott will share the link.
- Mixing models per sub-task (a frontier planner, cheaper models for vision/search sub-agents) is becoming normal — and it takes an engineer to design that mapping.
- Energy and environmental cost is a real concern.

## 3. Testing in 2026

### Why testing matters more now

There has "never been a better time to be a software tester." More code is being written with variable confidence in it; perhaps only 20–30% of AI-generated products going to production are tested to a sufficient level. Scott rates software testing as one of the safest career pathways in software engineering. (La Trobe has undergraduate and postgraduate software measurement and testing subjects, taught by Kathleen, a testing professional and ISO standards board member.)

Historic failures on the slides (the Pentium FDIV bug, the Morris worm, Mariner 1) were pre-AI engineering and requirements failures. Scott predicts the next edition of that slide will be full of **projects that failed through over-reliance on untested AI code** — "come back and see me in five years."

Testing is critical where fixes are expensive or lives are at stake: embedded systems and micro-controllers without an easy upgrade path (an on-site firmware reflash costs more than the installation earned), process control, production lines, military, medical and safety-critical systems. **Fixing bugs in production costs far more than fixing them in development.**

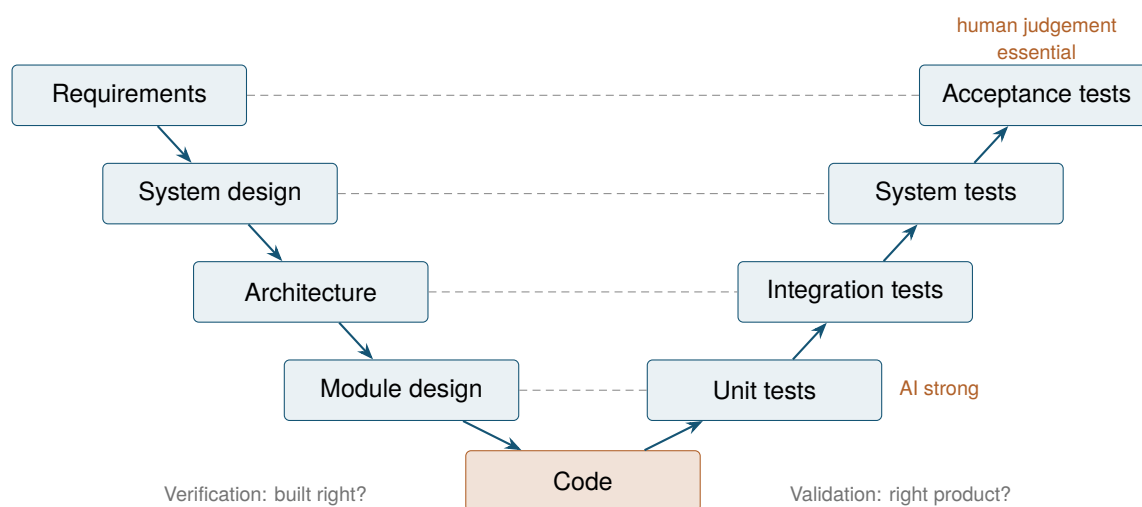
### Testing versus debugging

|        | Testing                                                                       | Debugging                                                   |
|--------|-------------------------------------------------------------------------------|-------------------------------------------------------------|
| Nature | The <b>plan</b> — how the system should behave and where failures could arise | The <b>action</b> — locating the fault                      |
| Tools  | Test plans, acceptance criteria, automated suites                             | Breakpoints, inspecting runtime memory state, AI assistance |

The vibe-coding anti-pattern: paste the terminal error into an AI, accept its wrong diagnosis, and chain further wrong answers. Instead, let the **segmented test plan** tell you *where* the fault logically is, then debug there with breakpoints and AI as an assistant. Keep the breakpoint skill.

### Verification, validation and where AI fits

- **Verification** — *are we building the product right?* Static analysis, style checks, proofs of correctness, consistency checks, unit and integration tests. These are rule-based, so **AI is strong here**.
- **Validation** — *are we building the right product?* Prototypes, demos, UML, model checking, inspections, and going back to the project owner. When AI builds a runnable prototype, a human still has to judge whether it does the right thing.
- Unit tests are handled well by AI; integration, regression, usability and user acceptance tests involve progressively more human judgement that AI cannot fully replace.



The V model. AI assistance is strongest near the base (rule-based verification, unit tests) and weakest at the top (acceptance and usability).

Scott's challenge: take classic textbook diagrams like the V model and **work out where AI can add new capability to each stage** — that is what makes a 2026 developer stand out.

### Test-driven development and user-story test tables

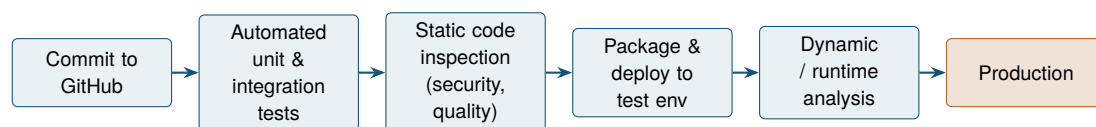
TDD: write the test first, write code that passes it, refactor, repeat, growing complexity each iteration. Many developers still skip straight to implementation, which **shifts the bottleneck** to later. Scott reminded the class that user stories with positive and negative acceptance criteria are a roadmap for testing — you understand what the system must do before writing code.

**Handover recommendation:** include a table like this for your user stories:

| Story | Test case | Acceptance criterion (normal / failure) | Expected behaviour                | Result |
|-------|-----------|-----------------------------------------|-----------------------------------|--------|
| US-01 | TC-01.1   | Normal: valid input                     | System does X                     | Pass   |
| US-01 | TC-01.2   | Failure: invalid input                  | Handles gracefully, clear message | Fail   |

Also add **sequence diagrams** for key user stories to show data and control flow across components. “The project owners will love it, so will I, and the marking system definitely supports that.” A dictionary aligning functions, methods and tests is a bonus for complex systems, and AI is good at generating that documentation from a project directory — but check it by hand.

### Automation and CI/CD



Every commit triggers the pipeline; each gate is a testing opportunity.

Testing weekly is “old school”; the target is testing on every commit. The main win is that issues surface much sooner because tests run more often. Every language and framework has test tooling; pick one, wire it in, run, analyse, proceed.

### Key takeaways from the testing slides

1. Design specific tests for each user story against its acceptance criteria — **both** normal behaviour and graceful handling of abnormal states (no crashes, no raw error dumps to users).
2. Check tests against the acceptance criteria **and** the sequence diagram.
3. A comprehensive dictionary of methods and tests helps maintain clarity.
4. Research automated testing and make informed choices, but accept that **coverage** is the challenge — firewalls, legacy hardware and software may block automation. Automate what you can.
5. **Security sign-off stays human.** AI can accelerate a first-pass audit against standards such as the OWASP Top 10 (broken access control, security misconfiguration, supply-chain failures, cryptographic failures, injection, insecure design...), but AI cannot sign a responsibility statement or be held accountable. Build secure by design, then have a human expert evaluate.
6. **Pair programming** — with a human or an AI “pair programmer” (Copilot’s own marketing) — produces more robust testing. Some collaboration beats none.
7. Use TDD to guide both testing and development.

## Action items

---

- ☐ Contact the project owner(s) this week to confirm direction before the handover phase.
- ☐ Use the LMS responsibility tracker if the team has sentiment or responsibility issues.
- ☐ Start an “AI assistance” workbook (one tab per project, aggregate chart); log IDP as project one.
- ☐ Start a weekly AI learning diary: what worked, what wasted time, what prompt changes helped.
- ☐ Build a user-story test table (normal and failure criteria, pass/fail) for the handover report.
- ☐ Draw sequence diagrams for the key user stories.
- ☐ Wire automated tests into GitHub CI so every commit runs them.
- ☐ Next week: effective reporting — final report and presentation materials.

## Observations for the LJA project

---

- **Model selection and the tender’s LLM cost.** Scott’s “frontier models are often unnecessary” point directly supports justifying a mid-tier model in the LJA’s cost estimate. A small side-by-side evaluation of two tiers on the rubric-to-SILO mapping task would be a strong, evidence-based addition to the compendium.
- **The synthetic-data harness is a test oracle.** Because `truth_theta.csv` and `truth_gaps.csv` exist, gap detection can be tested against known ground truth — exactly the kind of validation Scott says AI cannot do for you. Reporting the 0.49–0.63 correlation range across seeds as a tolerance band also mirrors his control-chart framing.
- **User-story test tables and sequence diagrams.** The extract → semantic map → gap detection → dashboard flow is a natural sequence diagram, and each stage has clear normal/failure acceptance criteria (e.g. an assignment with no rubric, a criterion with no SILO match).
- **Security-first alignment.** The OWASP discussion fits the subject’s DevSecOps mandate; the student-data handling in the Moodle plugin (access control, injection) is where an OWASP-based first-pass audit would add most.
- **Personal metrics.** The AI-assisted development on this project is a ready-made first tab for the workbook Scott described.